

LAPORAN PRAKTIKUM PEMOGRAMAN BERORIENTASI OBJEK

Pekan 3

DI SUSUN OLEH :

DIGO YUANDRA

2511533017

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN LABORATORIUM : Dawi Syauqi



**DEPARTEMEN INFOTMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG**

2026

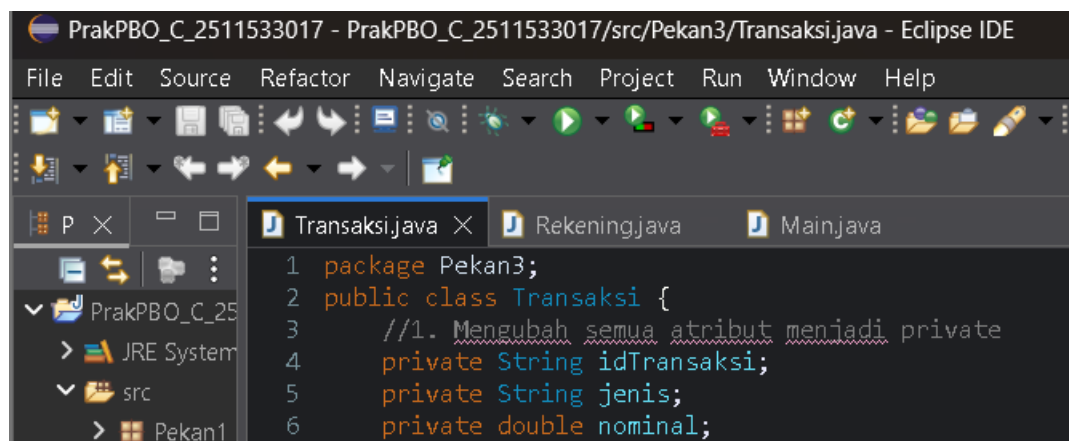
ISI

A.Laporan Praktikum

Pada langkah 1 :

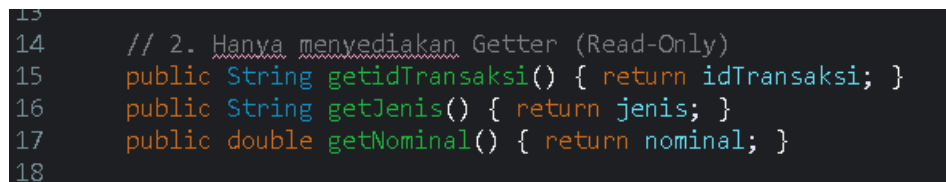
Bertujuan untuk mengunci entitas transaksi

1. Pada tahap pertama, file Transaksi.java yang telah dibuat pada praktikum sebelumnya dibuka kembali untuk dilakukan modifikasi. Perubahan dilakukan pada struktur class Transaksi dengan tujuan menerapkan konsep enkapsulasi pada atribut yang terdapat di dalam class tersebut.
2. Seluruh atribut yang terdapat pada class Transaksi, yaitu idTransaksi, jenis, dan nominal, diubah menggunakan access modifier private.



```
1 package Pekan3;
2 public class Transaksi {
3     //1. Mengubah semua atribut menjadi private
4     private String idTransaksi;
5     private String jenis;
6     private double nominal;
```

3. Setelah atribut dibuat menjadi private, dibuat method getter untuk memberikan akses membaca data transaksi secara terbatas.



```
13
14 // 2. Hanya menyediakan Getter (Read-Only)
15 public String getIdTransaksi() { return idTransaksi; }
16 public String getJenis() { return jenis; }
17 public double getNominal() { return nominal; }
18
```

Getter digunakan sebagai penghubung antara atribut private dengan class lain yang membutuhkan informasi transaksi. Dengan adanya getter, data masih dapat ditampilkan tanpa harus membuka akses langsung terhadap atribut. Pada implementasi ini, hanya data yang diperlukan saja yang diberikan akses publik sehingga informasi yang bersifat internal tetap terlindungi.

4. Pada tahap terakhir, dibuat method cetakDetail() yang berfungsi untuk menampilkan informasi detail transaksi.

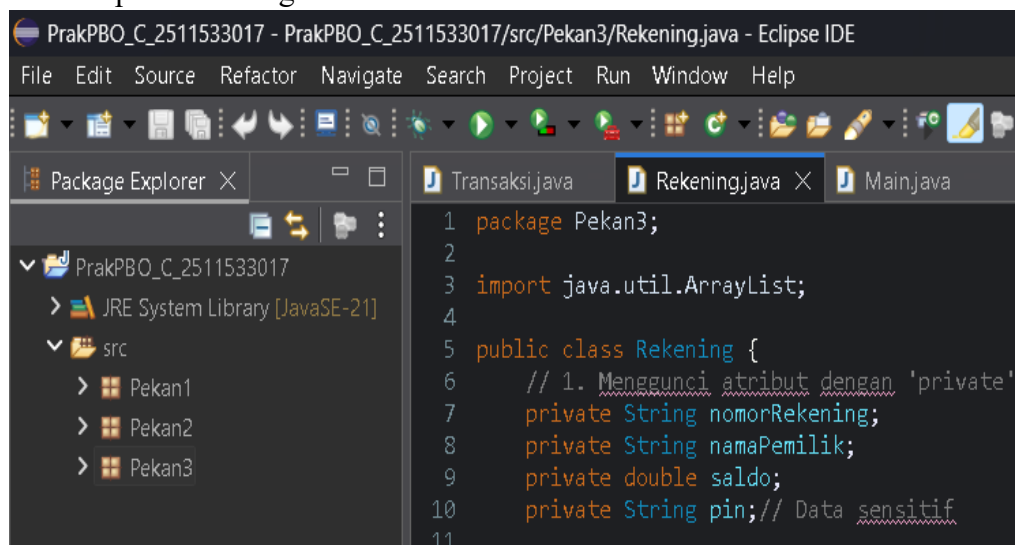
```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal : Rp" + nominal);  
}  
}
```

Method ini digunakan agar proses pencetakan data transaksi dilakukan melalui fungsi yang telah dibuat di dalam class Transaksi. Dengan cara ini, class lain tidak perlu mengakses setiap atribut secara langsung ketika ingin menampilkan informasi transaksi. Selain membuat program lebih terstruktur, penggunaan method ini juga membantu menjaga keamanan data karena seluruh proses pengolahan data tetap dikendalikan oleh class pemiliknya.

Langkah 2 :

Pada langkah 2 dilakukan penguncian entitas Rekening dan menambah PIN

1. Pada tahap ini, file Rekening.java yang telah dibuat pada praktikum sebelumnya dibuka kembali untuk dilakukan modifikasi. Perubahan dilakukan pada class rekening dengan tujuan meningkatkan keamanan data rekening menggunakan konsep enkapsulasi.
2. Seluruh atribut yang terdapat pada class Rekening diubah menggunakan access modifier private. Penggunaan modifier private bertujuan untuk membatasi akses langsung terhadap data rekening. Dengan adanya pembatasan ini, atribut seperti saldo dan PIN tidak dapat dimanipulasi secara langsung dari luar class.
3. Setelah seluruh atribut rekening diamankan menggunakan private, langkah berikutnya adalah menambahkan atribut baru yaitu pin. Atribut PIN digunakan sebagai sistem keamanan tambahan untuk melakukan proses autentikasi pengguna sebelum dapat mengakses fitur tertentu pada rekening.



4. Pada tahap ini konstruktor Rekening dimodifikasi agar dapat menerima input PIN saat pembuatan objek rekening baru.

Ketentuan yang diterapkan:

Jika PIN yang dimasukkan memiliki panjang 6 digit, maka PIN tersebut akan digunakan sebagai PIN awal rekening.

Jika panjang PIN tidak sesuai (kurang atau lebih dari 6 digit), maka sistem akan memberikan peringatan.

Sistem akan menggunakan PIN default yaitu 123456.

```
// Validasi PIN di dalam Constructor
if (pinAwal.length()==6) {
    this.pin = pinAwal;
} else {
    System.out.println("Peringatan PIN harus 6 digit! Menggunakan PIN default 123456");
    this.pin = "123456";
}
```

5. Setelah atribut dibuat private, dibuat method getter untuk atribut yang masih diperbolehkan diakses dari luar class, yaitu nomorRekening dan namaPemilik.

```
// 3. Getter untuk atribut yang diizinkan dibaca publik
public String getNomorRekening() { return nomorRekening; }
public String getNamaPemilik() { return namaPemilik; }
```

Getter digunakan agar informasi rekening yang bersifat umum tetap dapat ditampilkan tanpa membuka akses langsung terhadap atribut internal class. Pada implementasi ini, getter hanya diberikan untuk data yang aman untuk dibaca oleh pengguna. Sedangkan atribut seperti saldo dan pin tidak diberikan getter karena termasuk informasi yang harus dilindungi. Dengan pendekatan ini, sistem dapat menjaga keseimbangan antara kebutuhan akses data dan keamanan informasi rekening.

6. Pada tahap ini dibuat sebuah method otentikasi() yang berfungsi untuk melakukan proses pengecekan atau validasi PIN pengguna sebelum mengakses fitur tertentu pada rekening.

```
// 4. Method Otentikasi Internal (Validasi Enkapsulasi)
public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}
```

TUGAS

A.Tugas

Mengamati CLI dengan Otentikasi

1. Memperbarui Menu 1 (Buka Rekening) di kelas main, agar sebelum membuat objek user harus memasukkan PIN 6 digit terlebih dahulu

Pada tahap ini dilakukan modifikasi pada menu buka rekening di dalam class main. Sebelumnya, pengguna hanya perlu memasukkan data berupa nomor rekening, nama pemilik, dan saldo awal untuk membuat objek rekening baru.

Setelah dilakukan pengamanan menggunakan konsep enkapsulasi, sistem ditambahkan proses input PIN sebagai langkah keamanan tambahan. Pengguna diwajibkan memasukkan PIN awal dengan panjang 6 digit sebelum objek rekening dibuat.

Penambahan input PIN dilakukan menggunakan objek Scanner agar pengguna dapat memasukkan PIN melalui console.

```
if (sudahTerdaftar) {
    System.out.println("Error: Nomor rekening sudah terdaftar!");
} else {
    System.out.print("Masukan nama pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukan pin: ");
    String pin = input.nextLine();
    System.out.print("Masukan saldo awal: ");
    double saldo = input.nextDouble();

    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    System.out.println("Rekening berhasil dibuat dan langsung aktif!");
}
break;
```

Pada implementasi tersebut, nilai PIN yang dimasukkan pengguna akan diteruskan ke constructor Rekening. Selanjutnya sistem akan melakukan pengecekan apakah PIN memiliki panjang 6 digit atau tidak.

Jika PIN sesuai dengan aturan, maka PIN tersebut akan digunakan sebagai PIN awal rekening. Namun apabila PIN tidak memenuhi aturan, sistem akan memberikan peringatan dan menggunakan PIN default yang telah ditentukan.

Dengan penambahan proses ini, setiap rekening yang dibuat telah memiliki sistem keamanan berupa PIN sehingga akses terhadap fitur rekening dapat dikontrol dengan lebih baik.

Output:

PIN sesuai dengan instruksi (6 digit)

```
Menu Utama:
1. Buka Rekening Baru
2. Setor tunai
3. Tarik tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu : 1
Masukan nomor rekening: 1123
Masukan nama pemilik: 123
Masukan pin: 123456
Masukan saldo awal: 1000000
Rekening atas nama123 berhasil dibuat dengan saldo Rp1000000.0
Rekening berhasil dibuat dan langsung aktif!
```

PIN tidak 6 digit

```
Menu Utama:
1. Buka Rekening Baru
2. Setor tunai
3. Tarik tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu : 1
Masukan nomor rekening: 123
Masukan nama pemilik: 123
Masukan pin: 11
Masukan saldo awal: 1000000000
Peringatan PIN harus 6 digit! Menggunakan PIN default 123456
Rekening atas nama123 berhasil dibuat dengan saldo Rp1.0E9
Rekening berhasil dibuat dan langsung aktif!
```

2. Memperbarui Menu 3 (Tarik Tunai) dan Menu 6 (Cetak Mutasi), agar sebelum sistem menjalankan kedua method tersebut meminta user memasukkan PIN terlebih dahulu

Pada tahap ini dilakukan pengamanan terhadap fitur tarik tunai dan cetak mutasi dengan menambahkan proses autentikasi PIN sebelum sistem menjalankan method yang tersedia.

- Panggil fungsi akunAktif.otentifikasi(pinInput)
- Jika hasil autentikasi bernilai true, maka transaksi dapat dilanjutkan.
- Jika hasil autentikasi bernilai false, maka sistem akan menolak akses pengguna.

Tarik Tunai :

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening / belum login!");
    } else {
        System.out.print("Masukan pin: ");
        String pin = input.nextLine();
        if (akunAktif.otentifikasi(pin)) {
            System.out.print("Masukan nominal penarikan: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak : PIN yang Anda masukan salah!");
        }
    }
    break;
```

Berhasil

```
Menu Utama:
1. Buka Rekening Baru
2. Setor tunai
3. Tarik tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih Menu : 3
Masukan pin: 123456
Masukan nominal penarikan: 50000
Tarik tunai Rp50000.0 berhasil, Saldo saat ini: Rp9950000.0
```

Gagal (di tolak)

```
Pilih Menu : 3
Masukan pin: 112
Akses Ditolak : PIN yang Anda masukan salah!
```

Cetak Mutasi :

```
case 6:
    if (akunAktif == null) {
        System.out.println("Error : anda belum membuka rekening / belum login");
    } else {
        System.out.print("Masukan pin: ");
        String pin = input.nextLine();
        if (akunAktif.otentikasi(pin)) {
            akunAktif.cetakMutasi();

        } else {
            System.out.println("Akses Ditolak : PIN yang Anda masukan salah!");
        }
    }
    break;
```

Berhasil :

```
0. Keluar
Pilih Menu : 6
Masukan pin: 123456
ID: TRX-T1790698424748 | Jenis: Debit | Nominal : Rp500000.0
```

Gagal (PIN salah) :

```
Pilih Menu : 6
Masukan pin: 12354
Akses Ditolak : PIN yang Anda masukan salah!
```

Link github Praktikum :

https://github.com/Digooo112/PrakPBO_C_2511533017

